JAVA DOCS

Effective Java — Key Principles

Best Practices for Robust, Maintainable Java Code

Java Agentic AI — Knowledge Base Reference

1. Overview

This reference distills widely-adopted Java best practices (in the spirit of Joshua Bloch's well-known guidance) into core, actionable principles for writing robust, maintainable code. It focuses on the 'why' behind each rule so it generalizes beyond any single example.

2. Creating and Destroying Objects

2.1 Prefer Static Factory Methods Over Constructors

```
// Constructor - always creates a new object, name is fixed to the class
public Foo(boolean b) { ... }

// Static factory - can have a descriptive name, can cache/reuse instances, can return a
subtype
public static Boolean valueOf(boolean b) {
    return b ? Boolean.TRUE : Boolean.FALSE; // reuses cached instances, no allocation
}
```

- Static factories can have meaningful names (BigInteger.probablePrime(...) vs an ambiguous constructor).
- They aren't required to create a new object each call — enabling instance caching (like Boolean.valueOf).
- They can return an object of any subtype of the return type, hiding implementation classes entirely.
- The main downside: classes with only static factories and no public/protected constructor can't be subclassed.

2.2 Builder Pattern for Many Constructor Parameters

When a class would need many optional constructor parameters ('telescoping constructors'), the Builder pattern (see Design_Patterns.pdf for full code) reads far more clearly than a long parameter list where argument order is easy to get wrong, especially for several parameters of the same type (e.g. multiple ints in a row).

2.3 Enforce Singleton with a Private Constructor or Enum

```
public enum Singleton {
    INSTANCE;
    public void doSomething() { }
}
// Preferred: immune to reflection-based instantiation attacks and serialization pitfalls
// that hand-rolled singletons must guard against explicitly.
```

2.4 Avoid Creating Unnecessary Objects

```
// Bad - creates a new String object every single call
String s = new String("hello");

// Good - reuses the pooled literal
String s = "hello";
```

```
// Bad - autoboxing in a hot loop creates millions of Long objects
Long sum = 0L;
for (long i = 0; i < Integer.MAX_VALUE; i++) { sum += i; }

// Good - use the primitive
long sum = 0L;
for (long i = 0; i < Integer.MAX_VALUE; i++) { sum += i; }
```

2.5 Eliminate Obsolete Object References

Even with garbage collection, holding onto references longer than necessary (e.g. a manually managed array-backed stack that doesn't null out popped elements) creates memory leaks. Whenever a class manages its own memory (like a custom cache or pool), be vigilant about obsolete references.

3. Methods Common to All Objects

3.1 Obey the equals() Contract

equals() must be: reflexive (x.equals(x) is true), symmetric (x.equals(y) == y.equals(x)), transitive (if x.equals(y) and y.equals(z), then x.equals(z)), consistent (repeated calls return the same result absent mutation), and never equal to null.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Point)) return false;
    Point p = (Point) o;
    return x == p.x && y == p.y;
}
```

Note: *A frequent bug: adding a new field in a subclass and overriding equals() there breaks symmetry with the superclass's equals(), since a superclass instance can never equal a subclass instance that has extra state to compare. Favor composition over extending a concrete class with equals().*

3.2 Always Override hashCode() When You Override equals()

```
@Override
public int hashCode() {
    return Objects.hash(x, y);
}
```

Note: *Equal objects MUST have equal hash codes. Skipping this breaks HashMap/HashSet: an object can be inserted successfully but then appear 'missing' on lookup because it hashes into a different bucket than an equal key used to search.*

3.3 Always Override toString()

A well-written toString() (concise, informative, all significant fields) massively improves debuggability — log statements and debugger displays become immediately useful instead of showing ClassName@1a2b3c.

4. Classes and Interfaces

4.1 Minimize Mutability

- Don't provide setters — make the class immutable where practical.
- Make all fields private and final.
- Ensure the class can't be subclassed (make it final, or use private constructors + static factories).
- Immutable objects are automatically thread-safe (no synchronization needed), simple to reason about, and safe to share/cache freely.

4.2 Favor Composition Over Inheritance

Extending a concrete class across package boundaries is fragile — the subclass depends on the superclass's implementation details, which can change in a new release and silently break subclasses in ways that violate encapsulation. Composition (wrapping the class as a private field and forwarding calls) avoids this coupling entirely.

4.3 Design and Document for Inheritance, or Prohibit It

If a class isn't explicitly designed to be safely extended (documenting exactly how its methods interact, especially any self-use of overridable methods), mark it final or make its constructors private. Undocumented, ad-hoc inheritance is a common source of fragile-base-class bugs.

4.4 Prefer Interfaces to Abstract Classes for Defining a Type

Since a class can implement multiple interfaces but extend only one class, interfaces are more flexible for defining a type/contract. Existing classes can be retrofitted to implement a new interface easily, but not to extend a new abstract class (single inheritance already used).

5. Generics

Full details are in Generics.pdf. Key Effective-Java-style guidance: don't use raw types in new code (they exist only for backward compatibility); eliminate unchecked warnings wherever possible since each one represents a potential ClassCastException at runtime; favor lists over arrays when generics are involved (arrays are covariant and not truly type-safe at runtime, while generic lists are invariant and fully checked).

```
// Use bounded wildcards to increase API flexibility (PECS: Producer Extends, Consumer
Super)
public void pushAll(Iterable<? extends E> src) { for (E e : src) push(e); }
public void popAll(Collection<? super E> dst) { while (!isEmpty()) dst.add(pop()); }
```

6. Enums and Annotations

```
// Prefer enums with instance fields/methods over int constants
public enum Operation {
    PLUS("+") { public double apply(double x, double y) { return x + y; } },
    MINUS("-") { public double apply(double x, double y) { return x - y; } };
```

```
    private final String symbol;
    Operation(String symbol) { this.symbol = symbol; }
    public abstract double apply(double x, double y);
}
```

Note: *This 'constant-specific method body' pattern replaces error-prone switch statements over int constants with type-safe, self-documenting code where the compiler forces every constant to provide its own behavior.*

7. Lambdas and Streams

- Prefer lambdas to anonymous classes for functional interfaces — lambdas are concise and lack 'this' ambiguity (a lambda's 'this' refers to the enclosing instance, unlike an anonymous class).
- Prefer method references to lambdas where they're more readable: Integer::parseInt over s -> Integer.parseInt(s).
- Use streams judiciously — overusing them for anything with complex control flow, checked exceptions, or heavy side effects can make code harder to read than an equivalent loop; readability should be the deciding factor, not stream-usage for its own sake.

8. Exceptions

- Use checked exceptions for conditions the caller can reasonably be expected to recover from; use unchecked (runtime) exceptions for programming errors.
- Favor standard exceptions (IllegalArgumentException, IllegalStateException, NullPointerException) over custom ones when they fit — reduces API surface a caller needs to learn.
- Document every exception a method can throw via Javadoc @throws, including unchecked ones that are especially likely.
- Include failure-capture information in exception messages (the invalid value, the expected range) to make debugging from a stack trace alone feasible.

9. Concurrency

Covered in depth in Java_Concurrency_in_Practice.pdf. Core Effective-Java-style guidance: synchronize access to any mutable state shared between threads; prefer higher-level java.util.concurrent utilities (ExecutorService, ConcurrentHashMap, CountDownLatch) over hand-rolled wait/notify code, since it's extremely easy to introduce subtle races or missed-signal bugs manually.

10. Quick Interview Recap

- Why prefer static factory methods over public constructors sometimes? They can have descriptive names, avoid creating redundant objects via caching, and can return any subtype — offering more flexibility than a constructor tied to exactly one concrete class.
- Why must equals() and hashCode() always be overridden together? Because HashMap/HashSet rely on hashCode() to select a bucket and equals() to confirm a match within it — inconsistency between the two silently breaks lookups.

- Why favor composition over inheritance? Inheritance across package/module boundaries couples a subclass to superclass implementation details that can change, violating encapsulation in ways composition avoids by only depending on the composed object's public API.
- Why are immutable objects easier to work with in concurrent code? They can't be modified after construction, so they require no synchronization to share safely across threads.